SEHGN: Semantic-Enhanced Heterogeneous Graph Network for Web API Recommendation

Xuanye Wang, Meng Xi, Ying Li, Xiaohua Pan, Yangyang Wu, Shuiguang Deng, *Senior Member, IEEE*, and Jianwei Yin

***Abstract*—With the growth of cloud computing, a large number of innovative mashup applications and Web APIs have emerged on the Internet. The expansion of technology and information presents a significant challenge to the discovery of Web APIs from multiple service ecosystems. Various Web API recommendation methods have been proposed for Mashup creation, but most either treat different feature factor interactions equally or solely rely on requirements for API recommendation. These approaches face several challenges such as API compatibility dependencies, ambiguous definition and boundary dilemmas of APIs, and sparse API invocation records. In this work, we propose a Semantic-Enhanced Heterogeneous Graph Network(SEHGN) for Mashup creation. To address the above deficiencies, we design a multi-semantic aggregator to capture semantic associations between features to encode multiple node-edge relationships. Then, we introduce a semantic embedding component to generate text embedding vectors for mashups and APIs to learn global and local semantic information about text documents at different levels of abstraction. Finally, we fuse the output vectors to obtain a list of candidate Web APIs. Experiences are performed on real datasets, and statistical results show that SEHGN outperforms state-of-the-art models in terms of overall and long-tail Web API recommendations.**

***Index Terms*—Web API recommendation, mashup creation, heterogeneous network, multi-semantic enhance, semantic embedding.**

I. Introduction

With the development of cloud computing and microservice architecture, cloud-native-centered application models are gradually becoming the first choice of users [1].

Manuscript received 17 October 2023; revised 19 March 2024; accepted 5 June 2024. Date of publication 21 June 2024; date of current version 9 October 2024. This work was supported in part by the National Key R&D Program of China under Grant 2023YFC3306303, in part by the National Natural Science Foundation of China under Grant 72174212, in part by the Regional Innovation Development Joint Fund of the National Natural Science Foundation of China under Grant U22A6001, and in part by the Key R&D Program of Zhejiang Province under Grant 2023C01213. *(Corresponding authors: Meng Xi; Ying Li.)*

Xuanye Wang is with the School of Software Technology, Zhejiang University, Hangzhou 310027, China (e-mail: xuanyewang@zju.edu.cn).

Meng Xi and Yangyang Wu are with the School of Software Technology, Zhejiang University, Hangzhou 310027, China, also with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, China, and also with the Binjiang Institute, Zhejiang University, Hangzhou 310027, China (e-mail: ximeng@zju.edu.cn; zjuwuyy@zju.edu.cn).

Ying Li, Xiaohua Pan, Shuiguang Deng, and Jianwei Yin are with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, China, and also with the Binjiang Institute, Zhejiang University, Hangzhou 310027, China (e-mail: cnliying@zju.edu.cn; panxh@zju-bj.com; dengsg@zju.edu.cn; zjuyjw@zju.edu.cn).

Digital Object Identifier 10.1109/TSC.2024.3417323

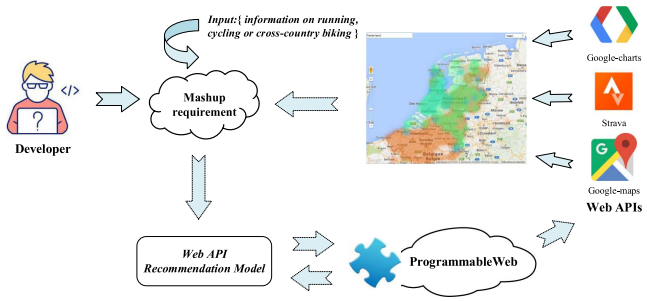

Fig. 1. An example mashup development process: Padvark.nl application integrates the Google Maps API, Google Charts API, and the Strava API to provide motion analysis services.

An increasing number of enterprises or organizations are publishing their business functions as remotely accessible APIs (application programming interfaces) to external developers and partners [2], which creates a favorable environment for the promotion of Mashup technology.

Mashups empower developers to create composite web applications by integrating and merging existing web API resources [3], which allows them to merge different applications, services, and data with each other according to their individual needs and creative requirements, resulting in new, rich and personalized web applications [4]. As shown in Fig. 1, developers plan to develop a motion analysis application where they can obtain the required information by entering a functional requirement description. This requirement information is passed and analyzed by trained models on the online API registry ProgrammableWeb.com to provide developers with candidate APIs that match each functional requirement. However, the proliferation of web APIs confers upon developers a saccharine burden in crafting mashups, as it brings forth constraints, limitations, and compatibility issues among the APIs. For instance, ProgrammableWeb has registered and published over 24,000 public APIs, each with unique features and functionality, which further adds to the complexity of API integration and management in Mashup creation.

Various Web API recommendation methods for automatic Mashup creation have been proposed, which are mainly divided into two categories: 1) Collaborative filtering (CF) based matching methods [5], [6]. In this type of work, it is common to share similar mashup experiences to enhance recommendation performance. The effectiveness of matrix factorization-based

1939-1374 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:37:05 UTC from IEEE Xplore. Restrictions apply.

methods depends largely on the availability of an adequate number of historical interaction records between Mashups and APIs. 2) Content-based matching methods [7], [8]. These works use various techniques to explore the relationship between requirements and Web APIs. Commonly, various features, including description text, tags, topics, popularity, and quality of service (QoS) are used in topic modeling methods [7] to improve the accuracy of recommendations. In recent years, an increasing amount of research has been conducted to improve the accuracy of Web API recommendations using network representation [9], [10] and deep learning [11], [12], [13]. Although these methods have achieved great success, their applicability in real-world mashup applications is limited:

1) *API compatibility dependency*: Mashup services often rely on multiple APIs, but not closely similar APIs are suitable for use in the same [14]. Specifically, there may be differences in implementation and maintenance between API providers, which can have an impact on the consistency of APIs that implement similar functionality, even if their relevance can be determined through textual descriptions. As a result, APIs with similar functionality are more likely to be viewed as substitutes rather than partners in Mashup.

2) *Ambiguous definitions and boundary dilemmas of APIs*: Natural language ambiguities often lead to differences in the interpretation of API documentation, which can result in the inputs, outputs, or behavior of the actual API not being clearly or consistently defined [15]. Specifically, It is often the situation in teamwork that different developers will write an API document at the same time, further adding to the complexity of defining and understanding ambiguity in application interfaces. Similarly, each system has its own unique API design style and naming conventions, especially in terms of API design, where the positioning of different APIs is often easily confused and misunderstood because of the lack of uniform standards.

3) *Sparse API invocation records*: Sparse API invocation records unintentionally delay tail APIs, with only 1.8% of mashups covering 99% of APIs [16]. Over-reliance on popular APIs can lead to a variety of problems, including reduced adaptability and scalability as well as service vulnerability. Specifically, a mashup service is overly reliant on a small number of major popular APIs, and when these APIs fail or become unavailable, the entire service may not function properly, making the service very sensitive to a single point of failure and lacking fault tolerance and reliability. Similarly, it also can make it difficult for the mashup service to adapt to new APIs or meet new requirements. Since popular API input and output parameters are stationary, it tends to limit the flexibility and scalability of the Mashup service, thus making it difficult for the service to evolve and improve. Therefore, it is impractical and difficult to rely solely on API usage records to match and determine their intended purpose.

In this work, we develop a **S**emantic-**E**nhanced **H**eterogeneous **G**raph **N**etwork(SEHGN) to improve the performance of the Web API recommendation method for Mashup creation. SEHGN is composed of two key components: One is the Multi-Semantic Aggregator(MSA), which encodes the underlying structural relationships in *heterogeneous graph of Mashups and APIs(MA-HG)* and performs multidimensional semantic enhancement by utilizing rich contextual semantic attribute information. The other is the Semantic Embedding Component (SEC), which efficiently combines the contextual semantic relationships of text documents through the attention mechanism to deeply explore the requirements of users and the functional descriptions of APIs. We emphasize the critical importance of leveraging MA-HG, with emphasis on mining information about collaboration between APIs and the value of tail APIs in pursuit of suitable and diverse API compositions. Comprehensive experiments on real datasets show that SEHGN is effective and outperforms the other state-of-the-art models for Web API recommendation. The main contributions of this work are summarized as follows:

- We propose a Semantic-Enhanced Heterogeneous Graph Network (SEHGN) for mashup creation which seamlessly integrates textual semantic contexts and diverse network relationships, as well as dynamically adaptively updating node weights to satisfy the mashup requirements.
- We propose a semantic embedding component to construct different levels of abstraction for textual descriptions, which alleviates document sparsity and semantic boundary ambiguity by using attention mechanisms to deeply explore contextual semantic dependencies in textual documents, and generating functional semantic vectors for representing Mashups and APIs.
- We propose a novel modeling approach called Heterogeneous Graph between Mashups and APIs (MA-HG), which integrates affinity graphs between APIs, compatibility graphs between APIs, bipartite graphs of Mashups and APIs, as well as their contextual semantics. It connects tail Web APIs to the backbone topology of the heterogeneous graph, unleashing the unexploited potential of tail Web APIs in the composition, and ultimately alleviating the challenges of long-tail distribution.
- Based on MA-HG, we propose the multi-semantic aggregator, which mainly consists of a multi-layer light graph convolutional network that can integrate various node-edge relationships and contextual attribute features to enhance the representation of semantic features. It encodes multivariate structural relationships, which can effectively capture valuable perceptual relationships between nodes in heterogeneous network embeddings, and amplify the local implicit relationships of structural signals inherent in the topology.
- We conducted comprehensive experiments to evaluate the proposed approach using real datasets. The experimental results demonstrate the superiority of the SEHGN approach over other state-of-the-art models both in terms of overall and long-tail recommendations. Specifically, compared to the baseline optimal metrics, SEHGN improves by at least 5.2% in NDCG, 3.7% in MAP, 2% in Precision, and 6% in Recall.

The rest of this work is organized as follows. Section II provides some background knowledge about the service ecosystem and proposes the construction of MA-HG. Section III presents the technical details of our approach. Section IV conducts

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:37:05 UTC from IEEE Xplore. Restrictions apply.

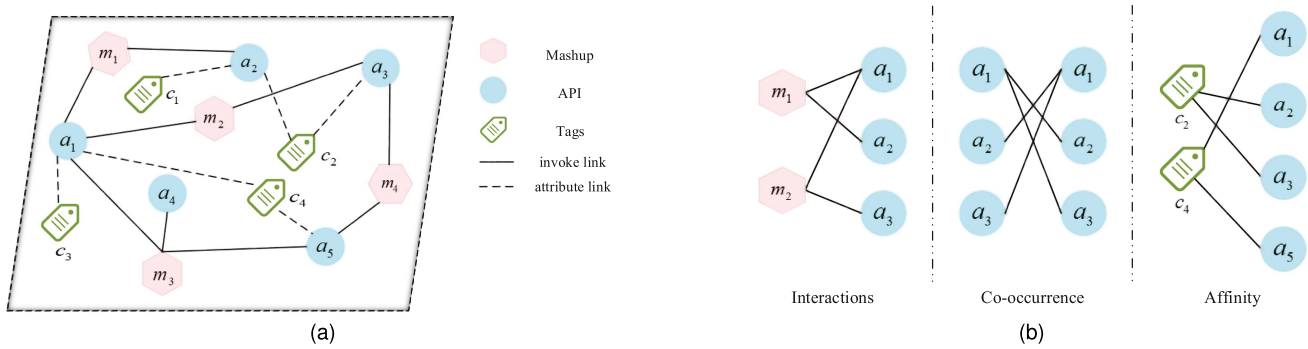

Fig. 2. Modeling of MA-HG: an example. (a) shows the construct of MA-HG. (b) shows the heterogeneous network relationship in MA-HG.

experiments and results analysis. Section V offers the related work on Web API recommendation technologies. Section VI concludes our work.

II. Problem Formulation

In this section, we formally state the problem to be studied in this work. First, we introduce concepts related to the service ecosystem. Then, based on these concepts, we model such heterogeneous information between Mashups and APIs as a heterogeneous graph. Finally, we give a generalized formulation of the Web API recommendations.

A. Service Ecosystem

In a large service ecosystem, Mashup applications are comprised of a set of Web APIs, formally represented as $E = (\mathbb{A}, \mathbb{M})$, where $\mathbb{A} = \{a_1, a_2, \ldots, a_{|\mathbb{A}|}\}$ and $\mathbb{M} = \{m_1, m_2, \ldots, m_{|\mathbb{A}|}\}$ denote the collections of Web APIs and Mashups in the ecosystem, respectively. Each new Mashup is related to sparse attributes denoted as $m_i = \{D_{mi}, T_{mi}, C_{mi}, A_{mi}\}$. where D_{mi} is a textual description used to introduce function information, T_{mi} is a set of tags used to indicate its categories C_{mi}, and A_{mi} is a collection of API used to indicate participation in mashup applications. Similarly, the set of sparse attributes in terms of each Web API is defined as $a_i = \{D_{ai}, T_{ai}, C_{ai}\}$, where D_{ai} is a text description used to introduce function information, and T_{ai} is a set of tags used to indicate its categories C_{ai}. These APIs encapsulate their functionality within API text documents, which are essential for meeting the requirements of the respective Mashup applications.

B. Heterogeneous Graph Between Mashups and APIs

As shown in Fig. 2(a) and (b), a heterogeneous graph between Mashups and APIs (MA-HG) is built for modeling Mashups, including Mashups, APIs, and associated attribute data based on abundant relationship information, such as the relational data among Mashups, APIs, and their properties: i.e., Mashup-API bipartite graph information, API-API co-occurrence graph information, and API-API affinity graph information. In the MA-HG $G = (V, E)$, V represents various kinds of nodes and E represents various relationships between nodes, where $V = \mathbb{A} \cup \mathbb{M}$ and $E = E_i \cup E_c \cup E_a$. E_i represents interaction relationships/connections between Mashups and APIs. E_c represents co-occurrence relationships/connections between APIs and APIs. E_a represents affinity relationships/connections between APIs and APIs. Note that APIs with co-occurrence are better suited to appear in the same Mashup, while APIs with affinity are better suited to be substituted for one another.

C. Problem Definition

Given an MA-HG G and a functional request for a mashup described by a sequence of keywords $DW = \{q_1, q_2, \ldots, q_N\}$, where q_i means keywords and N is the total number of keywords. The goal of Web API recommendation is to build a model that can generate different compositions of Web APIs, providing developers with recommendations for the top-N APIs as accurately as possible.

III. Method Overview

In order to handle the challenges in Section I, we propose a well-designed framework, Semantic-Enhanced Heterogeneous Graph Network (SEHGN), for Web API recommendation. Our framework, as demonstrated in Fig. 3, comprises three key components: Semantic Embedding Component(SEC), Multi-Semantic Aggregator(MSA), and Web API Recommendation Component(WARC). Proper feature representation is the cornerstone of deep network models.

- Within the SEC, we are aware of the sparsity of documents and construct different levels of abstraction for textual descriptions. By utilizing the attention mechanism, we deeply explore the contextual semantic dependencies in the textual documents to generate vectors of functional semantic representations for mashups and APIs.
- Within the MSA, we decouple MA-HG into multiple homogeneous and bipartite graphs to efficiently deal with the streaming relationships within the heterogeneous network, and then, we enhance the graph convolution process by exploiting the contextual semantics and feature relationships to explore the importance of different APIs in light graph convolution and subsequently re-aggregate the subgraphs.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:37:05 UTC from IEEE Xplore. Restrictions apply.

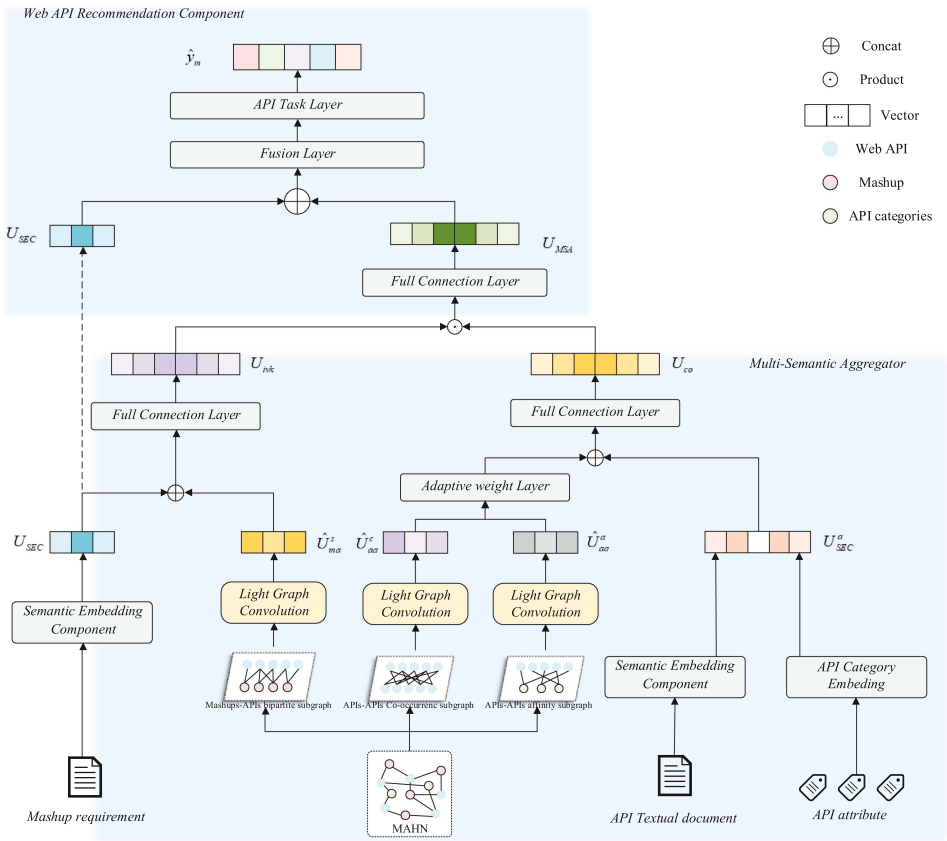

Fig. 3. Illustration of the proposed SEHGN.

- Within the WARC, we adopt a feature fusion approach to integrate the output vectors of these components to recommend a curated list of candidate Web APIs.

A. Semantic Embedding Component

The basic component of text representation in the SEC is the self-attention network (SAN) [17]. As shown in Fig. 4(a), the SEC serves as a pivotal element for acquiring contextual semantic representations of textual documents, which combines different linear or non-linear features to obtain global and local semantic representations of the text document.

In the self-attention layer, the word embedding vectors of the requirement text E_m are used to perform attentional operations by sliding a word window of size ws_j to establish associations with the preceding and following word vectors. Specifically, for each word, the self-attention mechanism computes its correlations with other words in the window and uses these correlations as weights to weight the representation of the corresponding word.

$$g_i(DW) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V \tag{1}$$

where Q, K, and V respectively represent the query, key, and value matrices obtained through matrix multiplication transformation, d_k represents the dimension of the word vector, and softmax represents the activation function which is used to calculate attention weights. In this work, Q, K, and V are multiplying the input sequence DW by three matrices W^Q, W^K, and W^V, respectively.

The weighted word representations are aggregated to be used in generating the final context-sensitive representation.

$$G_i(DW) = \sum_{i=1}^{n} g_i(DW) \cdot E_i(DW) \tag{2}$$

where $E_i(DW)$ represents the weighted word representation of the i-th word. After that, each word embedding vector generates a semantic feature vector G_i. These semantic feature vectors are subjected to Max-Pooling [18] to extract significant features and eliminate redundant details.

$$\mathcal{E}_j = [\text{Max}Pooling(G_i); \ldots; MaxPooling(G_I)] \tag{3}$$

where I is the number of word lengths. Generally, feature extraction can be enhanced by using windows of different sizes. Hence, the extracted features are concatenated to form a comprehensive feature representation $\mathcal{E} = [\mathcal{E}_1; \ldots; \mathcal{E}_j; \ldots; \mathcal{E}_J]$ and fed into a fully connected layer.

$$U_{SEC} = sigmoid(W_0\mathcal{E} + b_0) \tag{4}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:37:05 UTC from IEEE Xplore. Restrictions apply.

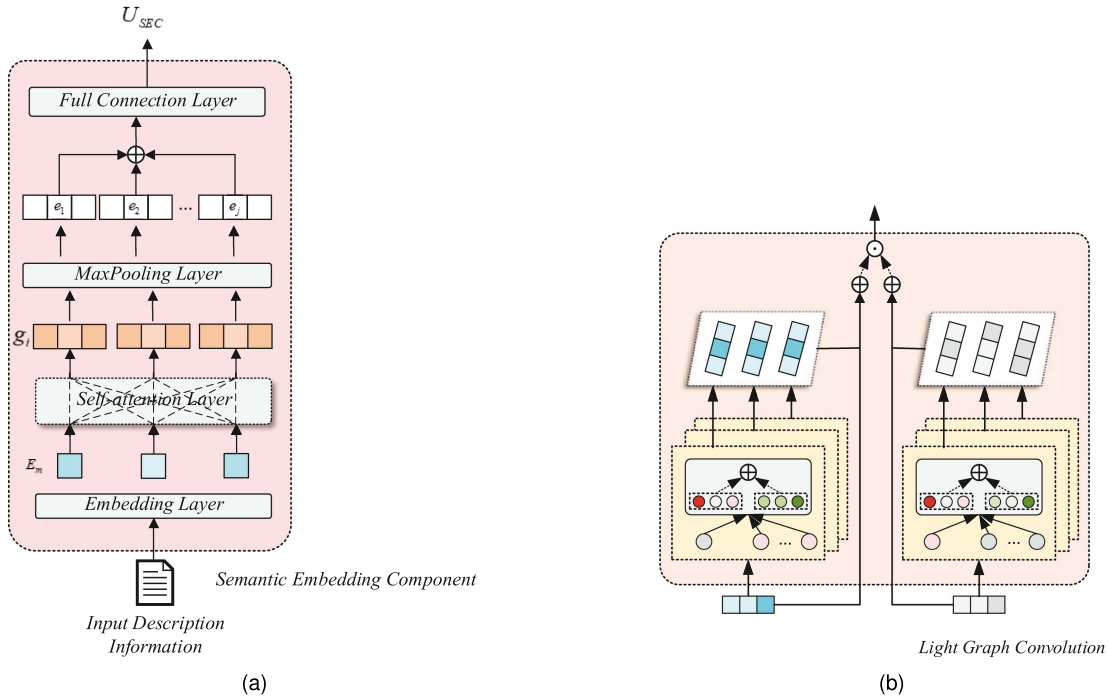

Fig. 4. The Semantic embedding component and light graph convolution of SEHGN. (a) shows the Semantic embedding component. (b) shows the light graph convolution.

where W_0 is the weight matrix, b_0 is the bias term, sigmoid is the activation function and e_J is the results of the word window with a size of ws_j.

In order to simplify the expression, we rewrite it as $U_{SEC} = SEC(\theta, m)$. where $SEC(\theta, m)$ is the function to encode the text m and parameter $\theta = \{\mathcal{E}_*, G_*, W_0, b_*\}$.

B. Multi-Semantic Aggregator

The rich structural information in MA-HG plays a crucial role in discovering the hidden patterns and relationships in the graph. As defined in Section II, MA-HG is a multi-attribute heterogeneous graph that contains different node types, attribute relationships, and multiple edge types, each of which has different roles and impacts in node representation. Therefore, we can analyze and understand graph data in multiple dimensions, where different node types represent different entities or concepts with attribute relationships between them, and edge types describe how they are connected and interact with each other. It enables us to explore complex patterns and relationships between nodes, thus revealing hidden patterns in the graph, such as semantic association, dependency, and similarity.

In order to effectively aggregate node attributes and multiple edge types in node representations, MA-HG is employed as a strategy to decouple the graph into multiple isomorphic and bipartite graphs based on edge connectivity, where each subgraph can be analyzed more centrally based on specific edge types and attribute relationships, thus better focusing on the interactions and patterns associated with specific edge types and attributes. In addition, we employ adaptive aggregation of heterogeneous information to ensure more accurate node insights and representations that better reflect the local context in each subgraph.

Next, we take the binary graph of Mashups and APIs as an example to illustrate how our model captures multiple topological signals. For a single-layer light graph convolution(LGC) [19]:

$$e_m^{(k+1)} = \sum_{a \in N_m} p_{ma} e_a^{(k)} \tag{5}$$

$$e_a^{(k+1)} = \sum_{m \in N_a} p_{ma} e_m^{(k)} \tag{6}$$

where $p_{ma} = 1/\sqrt{|N_m||N_a|}$ is the Laplacian graph norm, N_m and N_a are the first-hop neighbour of mashup m and API a, respectively. $e_m^{(k)}$ and $e_a^{(k)}$ are the embeddings of mashup m and API a after propagation in the k_{th} layer, respectively.

The higher-order connectivity between mashups and APIs is crucial for encoding structural signals that are used to calculate correlation scores. Specifically, we perform k LGC operations on the embedding of 0_{th} layer to obtain the embedding representation of the k_{th} layer and aggregate the embedding representation obtained from each layer as follows:

$$e_m = \sum_{k=0}^{K} \alpha_k e_m^{(k)} \tag{7}$$

$$e_a = \sum_{k=0}^{K} \alpha_k e_a^{(k)} \tag{8}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:37:05 UTC from IEEE Xplore. Restrictions apply.

where α_k denotes the importance of the k_{th} layer embedding in constituting the final embedding. The co-signals can be seamlessly injected into the presentation learning process by stacking multiple embedded communication layers.

Once the final embeddings of mashup m and API a are obtained, we can predict the user's preference by the inner product:

$$\hat{U}^s_{ma} = e_m^T e_a \tag{9}$$

The advantage of LGC is that it provides global semantics and abstract details in structural interactions. As the number of layers increases, the receptive field of each neuron becomes larger with respect to the previous layer, and the changes in the interaction that can be captured are more detailed, which is different from linear operations. Similarly, attribute relations are thought to play an important role in graph representation learning. Consequently, we aim to enhance the semantic representation of nodes by incorporating attribute relationships into the process of graph representation learning, which can be achieved as follows:

$$U_{ivk} = [U_{SEC}; U^s_{ma}] \tag{10}$$

where U_{SEC} is the semantic embedding of mashup m. The semantic enhancement of nodes, which concat with both, better helps to capture subtle relationships between nodes, enabling more accurate and insightful representations of nodes.

Meanwhile, with the multi-layered lightweight graph convolution and semantic enhancement, we also get the preference prediction of the corresponding inter-API isomorphic subgraph, which consists of both the compatibility and affinity subgraphs. In order to capture this multi-type of API node dependency, Relation-aware weights β^{sub}_{aa} are used to aggregate the isomorphic subgraphs among APIs as follows:

$$\hat{U}^s_{aa} = \beta_c \hat{U}^c_{aa} + \beta_a \hat{U}^a_{aa}; \beta^{sub}_{aa} = \{\beta_c, \beta_a\} \tag{11}$$

$$U_{co} = [U^a_{FSC}; U^s_{aa}], U^a_{SEC} = \left\{ \left[V^{SEC}_a; V^c_a\right] | a \in \mathbb{A} \right\} \tag{12}$$

where U^c_{aa} is the preference prediction of the compatibility subgraph between APIs, U^a_{aa} is the preference prediction of the affinity subgraph between APIs, V^{SEC}_a is the functional semantics of Web APIs, and V^c_a is the mapping vector of attribute features of Web APIs. Notice that the set of weights β^{sub}_{aa} should not be a set of hyperparameters but should be dynamically changed according to different tasks, so we set them as trainable parameters to be learned in train.

After obtaining the semantic enhancement preference predictions U_{ivk} and U_{co}, we get the final output of MSA:

$$U_{MSA} = U_{ivk} \cdot U_{co} \tag{13}$$

Here, we use the dot product operation to achieve a feature representation that is compatible with multiple relationships, helping us to capture the associations and interactions between features.

C. Web API Recommendation Component

In this subsection, we adopt a multi-model fusion technique. This technique involves combining different features or models to improve the generalizability and overall performance of the models. Based on the different dimension features provided by SEC and MSA, we perform an aggregation operation on these generated features as shown in Fig. 3.

$$\hat{y}_m = sigmoid(W_{TL} U_{MMF} + b_{TL}) \tag{14}$$

where $U_{MMF} = [U_{SEC}; U_{MSA}]$, W_{TL} and b_{TL} are the weight matrix and the bias vector in an API task layer, respectively.

We used binary cross entropy loss (BCELoss) to train the model parameters as follows:

$$Loss\left(\hat{y}_m, y_m\right) = \frac{1}{|A|} \sum_{a \in A} (y_m\left[a\right] \log\left(\hat{y}_m\left[a\right]\right) + \left(1 - y_m\left[a\right]\right) \log(1 - \hat{y}_m\left[a\right])) \tag{15}$$

where $y_m[a]$ represents the target value indicating whether or not the a-th Web API satisfies the requirements of mashup m, and $\hat{y}_m[a]$ denotes the predicted value corresponding to $y_m[a]$. To achieve the best fit for the target curve, we define an objective function in which L_2 regularization is applied to the parameters to prevent overfitting.

$$Loss\left(\theta'\right) = \min \frac{1}{|M|} \sum_{m \in M} Loss\left(\hat{y}_m, y_m\right) + \lambda \left|\left|\theta'\right|\right|^2_2 \tag{16}$$

where $\theta' = \theta \cup \{W_1, b_1, \ldots, W_z, b_z, V\}$ is the model parameters, and λ is a L_2 regularization weight.

In order to update the model parameters after each layer, we utilize a back-propagation algorithm to iteratively compute the gradients by recursively applying the chain rule, which allows us to efficiently compute the gradients and fine-tune the model parameters. Furthermore, to speed up training on large datasets, we utilize small batch Adam optimization, which not only accelerates convergence to the optimal solution but also efficiently handles noisy gradients.

After the model has been trained, we are able to generate a list of recommended candidate APIs for the mashup creation requirement DW_m by predicting the correlation score $\hat{y}_m \subseteq \{0, 1\}^{|\mathbb{A}|}$ and ranking all APIs based on that scores.

IV. Experiments

In this section, we introduce the experimental setup and analysis to evaluate the performance of our proposed approach in Web API recommendation, and the following research questions will be answered experimentally:

- RQ1: How is the performance of our proposed method SEHGN compared to common and state-of-the-art methods?
- RQ2: Does our proposed method SEHGN is competitive on long-tail distribution of APIs?
- RQ3: What is the contribution of semantic enhancement components and multi-semantic aggregators to the performance improvement of our proposed method SEHGN?
- RQ4: How do hyperparameters affect the performances, and how to determine the optimal values?

A. Experimental Settings

1) Datasets: We use a real-world dataset to conduct experiments to obtain solid experimental results. The dataset is

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:37:05 UTC from IEEE Xplore. Restrictions apply.

TABLE I
STATISTIC INFORMATION OF THE EXPERIMENTAL DATASET

Parameters	Values
Number of APIs	13176
Number of Mashups	8217
Number of API Invocations by Mashups	17182
Number of unique APIs used in Mashups	1647
Percentage of valid APIs	0.125
Sparsity of the Mashup-API matrix	0.1587 ‰
Average number of member APIs per Mashup	2.09
Average number of tags per APIs	2.447
Average number of tags per Mashups	2.271
Average number of terms in Mashup descriptions	17.019
Average number of terms in API descriptions	44.296

crawled from ProgrammableWeb.com, which contains historical interaction information between 8,217 mashups and 13,176 APIs. Among all the APIs, Only 1,647 Web APIs are invoked by Mashup, and 17,182 interactions are generated between Mashups and APIs. The sparsity of the Mashup-API matrix is defined as follows:

$$Sparsity = 1 - \frac{\#interactions}{\#Mashups \times \#services} \tag{17}$$

Thus, for commonly used APIs called historically in a Mashup, the mashup-API matrix has a dimension of 8217×13176, and its sparsity is 0.1587 ‰. More detailed information is provided in Table I.

2) Baseline Methods: There is no universal benchmark for Web API recommendation, so we have relied on a variety of representative recommendation methods as baselines to evaluate the efficacy of our proposed approach.

- *Traditional Collaborative Filtering (CF) methods*: These methods rely on the history of API invocations. Specifically, we evaluate Collaborative Filtering (CF) [5] and Neural Collaborative Filtering (NCF) [20] in our scenario.
- *Content-based methods*: These methods are usually based on the semantic features of the requirements. We consider Bi-directional Long Short-Term Memory(Bi-LSTM) [21] and Service Profile Reconstruction model (SPR) [8] as content-based methods. Bi-LSTM extracts the feature vectors of mashup description documents using the attention mechanism. whereas SPR uses topic modeling to discover important lexical features of APIs in the description and structure of mashups.
- *Popularity-based model (Pop)*: Pop is the most common baseline model, which takes the category-aware invocation frequency as the API usage hotness to rank the different APIs.
- *Random Walks with Restart (RWR)*: RWR [10] is centered around the use of a knowledge graph to encode the context and requirements of the mashup with categories. It recommends relevant APIs based on the relevance score obtained.
- *Multi-Task Fusion Model (MTFM)*: MTFM is a hybrid method that both models the semantics of requirements and generates latent interactions of history records [22].
- *Functional and Structural Fusion Model (FSFM)*: FSFM is a hybrid-based approach that models the semantics of requirements through convolutional operations and encodes a rich topology of structural interactions with operations such as graph convolution and matrix factorization [23].

3) Implement Details: In order to facilitate performance comparison and parameter analysis, we have employed the hold-out method to randomly divide the dataset into three distinct sets, namely the training set, validation set, and testing set. The complete dataset comprises a total of 6,573 Mashups, with an equal allocation of 822 instances for both the validation set and the testing set. As a default configuration, we have set the learning rate to 0.001 and used a training batch size of 128 in our experiments. The optimizer chosen for our model is Adam, and we applied a regularization weight λ of 1e-7 to prevent overfitting. Additionally, we implemented an early stopping strategy during the training phase to enhance efficiency.

For our model, in the SEC, the intercept length of the text is set to 50, the dimension of the pre-training word vector is fixed to 200, and the context window is set to the size of $\{2, 4, 8, 16\}$. In the MSA, the number of layers embedded K in the stack propagation layer is set to 3, and the embedding size of both Mashup and Web API is set to 128. Furthermore, to evaluate the impact of these parameters, we have examined the top-5 recommendation results, and similar results are obtained for other top-N recommendations. All experiments were conducted on a Linux system, utilizing the Pytorch deep learning library for seamless implementation.

4) Evaluation Metrics: There are numerous evaluation metrics for Recommender Systems (RS), and we have opted to utilize five commonly utilized metrics. i.e., Precision@N, Recall@N, MAP@N, and NDCG@N,. Precision@N and Recall@N are evaluation metrics that generate a single value without taking into account the order or ranking of items. In contrast, MAP@N and NDCG@N are order-aware metrics that consider the position of items and apply appropriate discounts to reflect their importance. In this experimental evaluation, we set N = 5, 10, 15, and 20, respectively. The metrics are formalized as follows:

$$Precision@N = \frac{|realAPI \cap topNAPIs|}{|topNAPIs|} \tag{18}$$

$$Recall@N = \frac{|realAPI \cap topNAPIs|}{|realAPIs|} \tag{19}$$

$$MAP@N = \frac{1}{M} \sum_{m \in M} \frac{\sum_{i=1}^{N} rel_i * Precision@i}{\sum_{i=1}^{N} rel_i} \tag{20}$$

$$DCG@N = rel_1 + \sum_{i=2}^{N} \frac{rel_i}{\log_2(i+1)} \tag{21}$$

$$IDCG@N = \sum_{i=1}^{|realAPIs|} \frac{2^{rel_i}}{\log_2(i+1)} \tag{22}$$

$$NDCG@N = \frac{DCG@N}{IDCG@N} \tag{23}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:37:05 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PERFORMANCE COMPARISON BETWEEN OUR METHOD AND OTHER APPROACHES ON OVERALL WEB API RECOMMENDATION

Method	N = 5				N = 10				N = 15				N = 20			
	NDCG	MAP	Precision	Recall	NDCG	MAP	Precision	Recall	NDCG	MAP	Precision	Recall	NDCG	MAP	Precision	Recall
CF^{2015}	0.2850	0.2178	0.1128	0.3273	0.3567	0.2408	0.0959	0.5414	0.3683	0.2397	0.0732	0.5958	0.3735	0.2386	0.0583	0.6277
NCF^{2017}	0.4209	0.3841	0.1323	0.3776	0.4501	0.3919	0.0833	0.4619	0.4600	0.3908	0.0615	0.5043	0.4655	0.3865	0.0499	0.5416
Pop^{2014}	0.4151	0.3781	0.1318	0.3749	0.4426	0.3853	0.0805	0.4450	0.4552	0.3847	0.0617	0.5072	0.4630	0.3823	0.0505	0.5484
$LSTM^{2019}$	0.4404	0.4020	0.1345	0.3875	0.4692	0.4078	0.0870	0.4779	0.4792	0.4052	0.0661	0.5324	0.4821	0.3996	0.0528	0.5610
SPR^{2016}	0.5723	0.5289	0.1791	0.5223	0.5955	0.5237	0.1133	0.6294	0.6036	0.5111	0.0887	0.7094	0.6071	0.5023	0.0734	0.7655
RWR^{2018}	0.6144	0.5709	0.1890	0.5651	0.6347	0.5698	0.1127	0.6474	0.6397	0.5611	0.0833	0.7000	0.6416	0.5543	0.0664	0.7345
$MTFM^{2021}$	0.6724	0.6447	0.2029	0.5784	0.6820	0.6341	0.1174	0.6504	0.6857	0.6255	0.0854	0.6956	0.6855	0.6178	0.0675	0.7211
$FSFM^{2023}$	0.7015	0.6817	0.2056	0.6079	0.7047	0.6710	0.1139	0.6481	0.7076	0.6664	0.0802	0.6708	0.7084	0.6614	0.0623	0.6879
SEHGN	**0.7634**	**0.7343**	**0.2258**	**0.6715**	**0.7647**	**0.7149**	**0.1311**	**0.7415**	**0.7637**	**0.7039**	**0.0935**	**0.7732**	**0.7605**	**0.6938**	**0.0734**	**0.7935**

where rel_i indicates whether the i_{th} Web API is truly relevant to the current Mashup or not. DCG metric computes the gain of the result list with decreasing discounts, and IDCG@N represents the ideal maximum value. Evaluation metrics are calculated in the test set to evaluate method performance.

Long-tail distribution problems arise when only a few popular APIs dominate the attention of most Mashups but the majority of APIs receive only little attention. A higher degree of hotness of a candidate API set will increase the burden of developing a Mashup. Over-reuse of the same APIs often leads to resource competition with other Mashups, thus limiting the performance and availability of the Mashup. Especially, with these popular APIs failing, changing, or going offline, Mashup may face even greater challenges. Therefore, we measure the long-tail distribution problem of candidate APIs based on their popularity bias as follows:

$$PB@N = \frac{1}{N}\sum_{i=1}^{N}\frac{hit_i}{ints} \tag{24}$$

where hit_i is the number of occurrences of the API in the history call log, and $ints$ is the total number of interactions generated between Mashups and the API. Obviously, $PB@N$ is in $(0, 1]$ after normalized by the number of all candidate APIs.

B. Experiment Results

1) Overall Performance Comparison(RQ1): As can be seen from Table II, hybrid-based methods(e.g., SEHGN, MTFM, FSFM) perform significantly better than traditional CF methods (e.g., CF, NCF), popularity-based methods (e.g., Pop), content-based methods (e.g., LSTM, SPR), and knowledge graph methods (e.g., RWR). Specifically, CF performs the worst due to not utilizing useful information. NCF performs better than Pop and CF due to capturing latent interaction factors. The content-based methods are significantly better than the traditional CF methods, which indicates the importance of considering developers' requirements for the Mashup, and demonstrates that textual description information carries a much greater wealth of knowledge compared to other metadata. The rich contextual semantic information maintains the semantic link between requirements and API descriptions to the maximum extent possible. In most cases, RWR is better than SPR, especially Recall, probably because the composition of tags is more descriptive of the developer's requirements, but it also makes it more difficult for the user to manipulate.

Obviously, the hybrid-based methods outperform the above methods. They fully utilize the potentially rich structural relationships among the composite APIs to learn the interaction patterns and potential associations. Benefiting from this mechanism, they can select suitable combinatorial APIs based on contextual semantic information dynamically. In the hybrid-based methods, the result that FSFM outperforms MTFM shows that explicitly encoding latent relationships between APIs is better than generating latent relationships through textual descriptions. SEHGN outperforms FSFM by 6.19% - 5.21% for NDCG, 5.26% - 3.75% for MAP, 2.02% - 1.11% for Precision, and 10% - 6% for Recall. It indicates that semantic enhancement helps mine potential interactions hidden in sparse topologies, deepening the characterization of Mashup and API nodes. In addition, we find that these methods are especially effective in recommending APIs when considering the top-5 and top-10 recommendations, which is because the average number of APIs used for Mashup creation in the dataset is less than 3. Therefore, as the number of recommendation APIs increases, the increase in Recall decreases gradually and the value of Precision decreases. Usually, users need less than 5 APIs to create a Mashup, so it is highly valuable to pay close attention to the APIs at the top of the recommendation list.

2) Long-Tail Performance Comparison(RQ2): Long-tail Web API recommendations refer to those Web APIs that are not popular and used by only a few mashups. In this subsection, we will use NDCG@N, Recall@N, and PB@N as representatives to evaluate the performance of our approach and other approaches on long-tail Web API recommendations, which obtain similar results in other metrics.

Comparing Fig. 5(a), (b), and (c), it can be seen that most methods perform differently on long-tail recommendations. Pop based on popularity shows a sharp decline in NDCG@N and Recall@N because it only recommends the most popular services in each category, ignoring the existence of tail APIs. Content-based methods focus only on functional descriptions and keywords, which makes them outperform traditional CF methods in long-tail recommendations. MTFM drops significantly in long-tail recommendations because MTFM uses text descriptions as input in interactive feature learning, which is inaccurate for features generated by tail APIs. Although FSFM and SEHGN demonstrate similar performance in terms of NDCG@N and Recall@N, SEHGN significantly outperforms FSFM on PB@N. This observation implies that employing semantics-enhanced heterogeneous graph convolution is beneficial when it comes

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:37:05 UTC from IEEE Xplore. Restrictions apply.

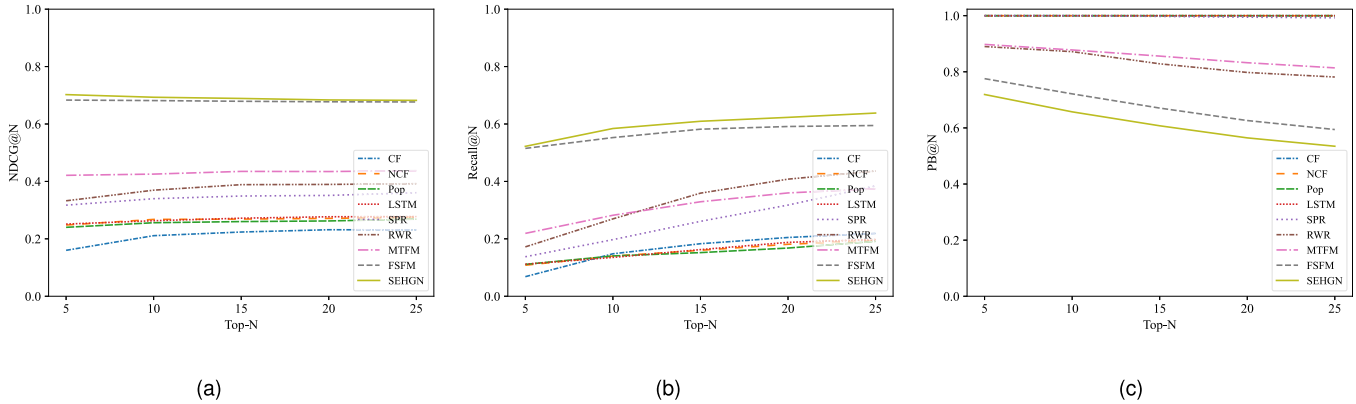

Fig. 5. Long-tail performance comparison between our method and other methods. (a) shows the NDCG@N results. (b) shows the Recall@N results. (c) shows the PB@N results.

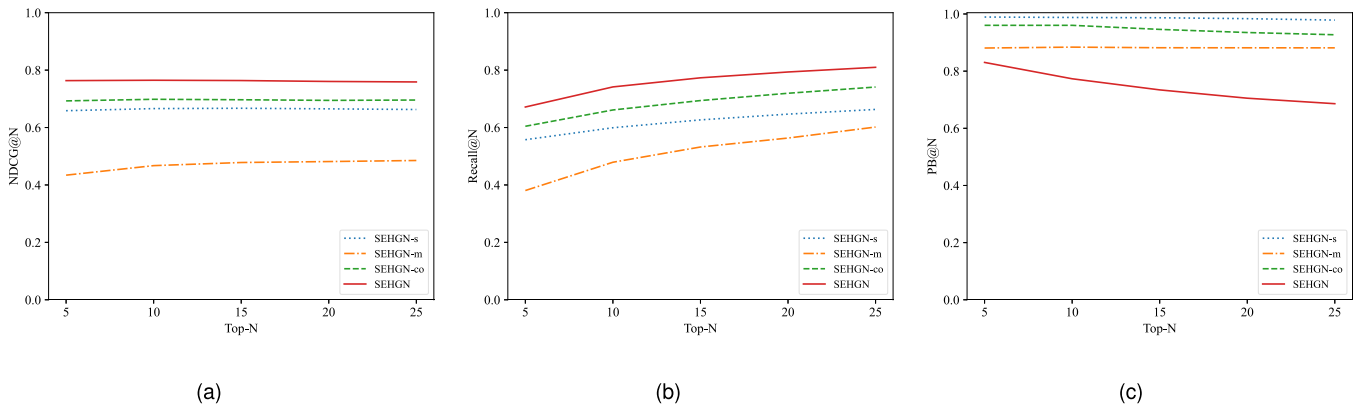

Fig. 6. Analysis of SEHGN components. (a) shows the NDCG@N results. (b) shows the Recall@N results. (c) shows the PB@N results.

to expressing less commonly used API features, which in turn offers increased flexibility in selecting personalized and diverse APIs. In addition, we find a change in the relation between RWR and MTFM in Recall when the range of N is from 10 to 15. In other words, MTFM outperforms RWR in terms of Recall@10, whereas RWR outperforms MTFM in terms of Recall@15. This is because the query weighting strategy and random wandering method of RWR improve the focus on the tail APIs, allowing it to match more real APIs among the top-10 to top-15 recommended lists.

C. Ablation Study

1) Analysis of SEHGN Components(RQ3): Ablation studies are a commonly used method in machine learning to assess the importance of individual components in models, especially in complex neural networks where it is crucial to understand the contribution of each component to the overall model performance. Therefore, we demonstrated its effectiveness by comparing the following variants of SEHGN:

- SEHGN-s: a variant of SEHGN that removes the multi-semantic aggregator and makes recommendations only through requirement semantics;
- SEHGN-m: a variant of SEHGN that removes semantic embedding components and makes recommendations only through MA-HG;
- SEHGN-co: a variant of SEHGN that removes isomorphic graphs in heterogeneous networks and makes recommendations only by combining historical interaction records and Mashup requirements.

As Fig. 6 shows the effect of SEHGN and its three variants, from which we can draw the following conclusions:

1) SEHGN-m removes the feature embedding component and basically degenerates into a heterogeneous graph convolutional network, which performs the worst on NDCG@N and Recall@N compared to the other three models, but is close to SEHGN on PB@N. It is attributed to the design of the MSA, which pays attention to the multivariate attribute features and heterogeneous connected-edge relations between Mashup and APIs. Specifically, it encodes the compatibility and affinity relationships in the API co-occurrence subgraphs and affinity subgraphs, especially in the case of tail APIs, which can provide unique information and even serve as a functional alternative to the head APIs, further enriching the diversity of recommendation systems.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:37:05 UTC from IEEE Xplore. Restrictions apply.

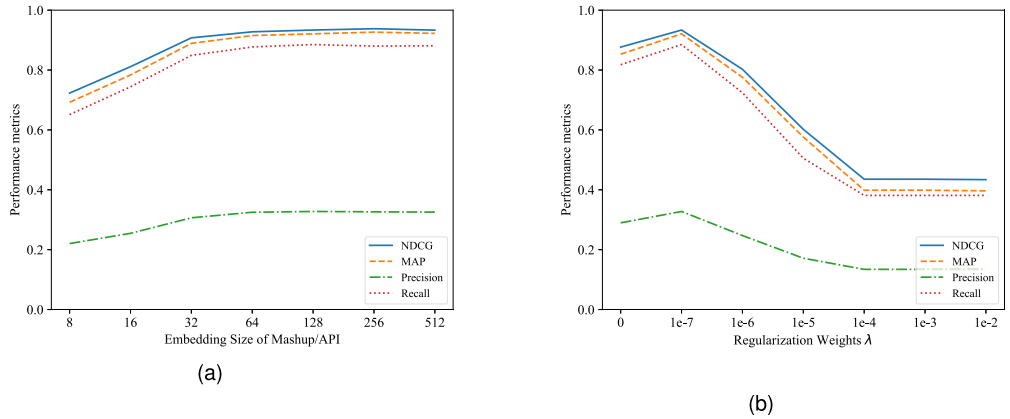

Fig. 7. Impact of embedding size and regularization weights. (a) shows the Impact of embedding size results. (b) shows the Impact of regularization weights results.

2) SEHGN-s can be viewed as a content-based approach that captures the correlation between the request Mashup requirement and the API functionality documentation through the attention mechanism, which brings at least 10% revenue on NDCG@N and Recall@N compared to SEHGN-m, which further highlights the value of the requirement semantics for the performance of recommender systems. The rich contextual content can effectively clarify the development requirements of mashups, maintain the semantic connections to the maximum extent possible, and alleviate the sparsity of structured interactions
3) SEHGN-co explores the collaborative relationship between Mashup and API by applying graph convolution operation to the bipartite graph of Mashup and API based on semantic embedding components to complement the semantic feature embedding. Compared to SEHGN-s, SEHGN-co brings a gain of 3% on NDCG@N and 5-7% on Recall@N. It indicates that collaborative relationships can better capture the underlying structural information and deepen the understanding and correlation between Mashups and APIs.

2) Impact of Embedding Size(RQ4): The embedding size determines the ability of the model to capture the attribute information of Mashups and Web APIs. We vary the embedding size in the set $\{8, 16, 32, 64, 128, 256, 512\}$. As shown in Fig. 7(a), we can observe that increasing the embedding size of Mashup and Web API initially improves the recommendation performance. It is reasonable that a larger embedding size can better contain different information about Mashup and Web API. However, the recommendation performance changes tend to be smooth when the embedding size exceeds 32, which may be due to the limited improvement of the features by increasing the embedding size. Moreover, It may lead to computational inefficiencies and the risk of overfitting when the embedding size is too large.

3) Impact of Regularization Weights(RQ4): In order to determine the optimal regularization level, we study the sensitivity of regularization weights λ by tuning this parameter within the range of $\{0, 1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2\}$, with the step size of $1e-1$. As shown in Fig. 7(b), the recommendation effect reaches the optimal value when λ is set to $1e-7$, which indicates that proper λ has a facilitating effect on the model. However, when λ exceeds $1e-7$, the model starts to find a balance between bias and variance, which leads to a decrease in performance on the test set. After λ exceeds $1e-4$, the regularization term becomes increasingly dominant, and the degradation in model performance tends to become smoother. This observation could be explained by the fact that the regularization term gradually restricts the model's output values to a narrower range, thereby diminishing the model's capacity to represent complex relationships and leading to under-fitting.

D. Case-Study Analysis

In this section, we show a running example and some actual cases of API recommendations in order to make a qualitative comparison of the selected methods, thus providing an intuitive impression of a real scenario. In particular, Table III shows the top five recommended results for the four target mashups, which cover different types of applications. A comparison with the ground truth shows that our method outperforms all the other methods. Compared to the other baseline models, SEHGCN perfectly fulfills the APIs needed for the requirements in the first three mashups, even in the face of a mashup containing six different APIs. For the mashup whats-on-twitter, SEHGCN also achieves the three target APIs in the top 5 recommendations.

E. Further Discussions

In general, developers can browse and evaluate potentially useful Web APIs in the repository during the application development process, and then utilize such tools to quickly integrate selected Web APIs into their applications. Thus, our approach SEHGCN can be used as a reference for application design and embedded in the mashup development cycle, after requirements analysis and specification and before the mashup composition, which aims to reduce the time and effort required by developers in exploring and discovering useful Web APIs. However, there

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:37:05 UTC from IEEE Xplore. Restrictions apply.

TABLE III
CASE-STUDY ANALYSIS ON API RECOMMENDATION WHERE THE APIS GET HIT ARE MARKED IN BOLD FONT

Mashup	Method	Top 5 Recommedations of API
ziitrend	CF	twilio, google-maps, facebook, twitter, **flickr**
	NCF	google-maps, twitter, youtube, **flickr**, amazon-product-advertising
	Pop	google-maps, facebook, twitter, youtube, **flickr**
	LSTM	google-maps, twitter, **flickr**, facebook, youtube
	SPR	google-maps, twitter, facebook, flickr, youtube
	RWR	google-maps, **flickr**, twitter, youtube, amazon-product-advertising
	MTFM	facebook, twitter, digg, **flickr**, youtube
	FSFM	facebook, twitter, del.icio.us, webthumb, **flickr**
	SEHGN	**flickr**, **google-adsense**, **google-search**, digg, facebook
	GT	**flickr**, **google-adsense**, **google-search**
spiggler-tweets-on-streets	CF	**facebook**, flickr, yahoo-maps, twitter, last.fm
	NCF	**google-maps**, **twitter**, flickr, youtube, **facebook**
	Pop	**google-maps**, youtube, flickr, **facebook**, amazon-product-advertising
	LSTM	**google-maps**, **twitter**, flickr, youtube, **facebook**
	SPR	**google-maps**, **twitter**, **facebook**, youtube, flickr
	RWR	**google-maps**, **twitter**, youtube, flickr, **facebook**
	MTFM	**google-maps**, **twitter**, **google-search**, **facebook**, google-app-engine
	FSFM	**google-maps**, **twitter**, **facebook**, foursquare, myspace
	SEHGN	**twitter**, **facebook**, **google-maps**, **google-ajax-libraries**, **google-search**
	GT	**facebook**, **foursquare**, **google-ajax-libraries**, **google-maps**, **google-search**, **twitter**
semantictweet	CF	amazon-product-advertising, twilio, last.fm, **twitter**, flickr
	NCF	google-maps, **twitter**, flickr, youtube, amazon-product-advertising
	Pop	google-maps, youtube, **twitter**, flickr, twilio-sms
	LSTM	google-maps, **twitter**, facebook, youtube, flickr
	SPR	**twitter**, google-maps, youtube, facebook, flickr
	RWR	google-maps, **twitter**, facebook, twilio, youtube
	MTFM	**twitter**, facebook, google-search, google-app-engine, wikipedia
	FSFM	**twitter**, twitpic, amazon-ec2, twitter-streaming, bit.ly
	SEHGN	**twitter**, **geonames**, facebook, flickr, wikipedia
	GT	**geonames**, **twitter**
whats-on-twitter	CF	flickr, amazon-product-advertising, google-maps, youtube, facebook
	NCF	google-maps, flickr, youtube, **twitter**, facebook
	Pop	google-maps, youtube, **twitter**, flickr, microsoft-bing-maps
	LSTM	google-maps, **twitter**, flickr, youtube, facebook
	SPR	**twitter**, google-maps, youtube, flickr, facebook
	RWR	google-maps, flickr, **twitter**, **google-homepage**, youtube
	MTFM	**twitter**, **google-homepage**, facebook, amazon-ec2, google-maps
	FSFM	**twitter**, twitter-search, yahoo-map-image, amazon-ec2, **google-homepage**
	SEHGN	**twitter**, **twitpic**, **google-homepage**, what-the-trend, yahoo-search
	GT	**google-homepage**, **tagalus**, **twitpic**, **twitter**

GT means ground truth.

are still several issues that need to be analyzed and discussed deeply as follows:

1) In this work, restricted keyword input may burden developers or users if they are not familiar with the background knowledge of the Web API. In order to solve this problem, we can relax the restriction of keyword input by front-loading the text pre-processing in the input, so that the developers or users can express their functional requirements for the mashup application with one paragraph or one sentence as the query input.
2) As defining the heterogeneous graph of Mashups and APIs in Section II, which simultaneously considers the interaction relationship between Mashups and APIs, the compatibility relationship between APIs, and the affinity relationship between APIs, which makes the time cost and space complexity of the model exponentially increase with the release of new APIs. Therefore, it is necessary to use some effective energy-saving techniques (e.g., [24]) in our application to improve the recommended efficiency, especially when the computational load is heavy enough.
3) In this work, we utilize mashups, APIs, and associated attribute data to build a heterogeneous network of Mashups and APIs, during which sensitive information of developers may be leaked. Therefore, it is necessary to refine our approach by adopting more effective data protection strategies(e.g., [25]) while ensuring Web API recommendation accuracy.

V. RELATED WORKS

The research related to Web API recommendation plays an important role in Mashup creation. The existing work is divided into three parts, i.e., collaborative filtering-based methods, content-based methods, and deep learning-based methods.

A. Collaborative Filtering-Based Methods

Collaborative filtering methods have been widely used as a common recommendation algorithm for Mashup creation. Cao et al. [5] propose a Mashup service recommendation method based on content similarity and collaborative filtering. Similarly, Chen et al. [26] combined user-based CF with item-based CF and proposed a QoS prediction method based on Web API, which helps users find appropriate Web services by analyzing QoS

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:37:05 UTC from IEEE Xplore. Restrictions apply.

information. Yu et al. [27] developed a tracking norm regularization matrix factorization algorithm to recommend services with the best QoS to users. However, this type of approach is limited by the fact that QoS information on APIs is incomplete and difficult to obtain. Botangen et al. [28] suggested a geographic-aware CF method for Web service recommendation, by integrating geographic information with functional descriptions to complement the Mashup-API invocation information. Zhang et al. [29] proposed a location-aware deep CF (LDCF) method, which maps location features into high-dimensional embedding vectors by a similarity adaptive corrector. However, it is difficult to use on a large scale considering the realities of information security and identity privacy. Rahman et al. [30] proposed a matrix factorization method of service cluster based on integrated content and network to improve the accuracy and diversity of the Web API recommendation results. Yao et al. [16], [31] proposed a probability matrix factorization with implicit correlation regularization to solve the problem of service recommendation and enhance the diversity of service recommendations. Fletcher et al. [6] incorporated the implicit user preferences (i.e., invocation history) into the matrix factorization model to improve the accuracy and diversity of recommendations.

CF-based methods leverage shared Mashup experiences and yield reasonable results even with sparse data. but may not prioritize the requirements or less popular tail APIs. Instead, we combine affinity graphs between APIs, compatibility graphs between APIs, bipartite graphs of Mashups and APIs, and their context attribute relationships to develop a complete end-to-end model for matching Web APIs and mashup requirements.

B. Content-Based Methods

Content-based methods rely on the matching between requirements and the functional description of the Web API. Xia et al. [7] proposed a category-aware service clustering and distributed recommendation method. Zhao et al. [32] proposed a Web API recommendation method using feature sets and learning-to-rank. They use mashup and Web API usage data, text features, nearest neighbor features, API-specific features, and API tag features to estimate the correlation between mashup requirements and API candidates. In contrast to the above approaches, Bao et al. [9] proposed a fine-grained API link prediction method to support mashup recommendations. Cao et al. [33] established a two-level topic model based on the relationship between Mashups and Web APIs. Shi et al. [34] proposed an attention-based relationship topic model, which takes tags as a priori information through the attention mechanism, mining web service text content and composition relationships to learn latent topics, and then predicting the latent link relationships between services.

The major hurdle for content-based methods is the significant amount of work involved in extracting valid features, which is often limited by the availability and quality of product functionality. On the contrary to these efforts, we propose a deep neural network-based modeling approach to build a complete end-to-end recommendation framework that automatically learns complex feature representations from raw data, reducing human intervention and quality dependence on textual data.

C. Deep Learning-Based Methods

With the development of deep learning techniques in various fields, several works use deep neural networks to solve the problem of Web API recommendation [21], [35]. For text description, Xue et al. [36] proposed that the recommended list of generated APIs should be further ranked according to whether the given API meets reasonable quantitative indicators. Shi et al. [21] proposed a recurrent neural network model based on tag attention for network service recommendation. They believe that the tag is part of the service description and assign attention scores to different words in the service description according to the importance of the text description intention. For the interactive structure, Xiong et al. [13] proposed a hybrid method based on deep learning, which combines collaborative filtering and text content to recommend web services. They seamlessly integrate the invocation interaction and their functions between mashups and services into a deep neural network, which can be used to describe the complex relationship between mashups and services. Kang et al. [11] propose a hybrid factorization machine, which captures the interaction of nonlinear features by integrating deep neural networks.

In addition, Wu et al. [22] proposed a neural framework based on multi-model fusion and multi-task learning. They use the convolution method to model text description information and the collaborative filtering method to extract collaborative information from the Mashup-API interaction matrix. Xiao et al. [12] proposed an API recommendation method based on structured enhancement and attribute weakening networks. They adaptively update the combination embedding features to meet the needs of different scenarios by introducing a deep interest network. Xie et al. [37] propose a generative response network (GAN) based service recommendation method to learn developer preference vectors by sampling meta-paths with different semantic relationships. Sang et al. [38] propose a Bayesian probabilistic matrix factorization based model. They used Doc2Vec technology and adversarial training to extract entire elements to mine requirement documents, mashup descriptions, and API descriptions, improving the robustness of the model generalization and maintaining the semantic relationships to the greatest extent possible. Similarly, several researches focus on providing a set of service bundles. Wu et al. [39] use API correlation graphs to model the functionality, popularity, and compatibility of Web APIs, and adopt Simhash and Hamming Distances to provide the most diverse compositions of APIs. Gong et al. [40] proposed a diversity-aware network-based recommendation approach, which modeled the recommendation problem as a graph search problem. They provide diverse service compositions by finding the smallest group of Steiner trees in a correlation graph. Gu et al. [41] proposed a compositional semantics-based service bundle approach that mines the semantic service package repository for bundle composite semantics to recommend service bundles that cover the functional requirements as completely as possible. Liu et al. [42] proposed a dynamic graph-based neural

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:37:05 UTC from IEEE Xplore. Restrictions apply.

network to model the constraints between services that address the evolution of services and the semantic gap between services and mashups.

These works have demonstrated that deep learning techniques can help develop more effective Web API recommendation models. Inspired by the above, we propose a heterogeneous graph MA-HG, trained by end-to-end supervised classification, and achieve positive results, which are significantly different from existing methods.

VI. Conclusion

In this work, a hybrid model based on deep neural networks(i.e., SEHGN), is proposed for Mashup creation, which seamlessly blends semantic attributes and network relationships. Through the multi-semantic aggregator, SEHGN encodes potential multiple structural relationships in a heterogeneous graph of Mashups and APIs by semantically enhancing various node-edge relationships and contextual attribute features. In addition, the semantic embedding component generates text embedding vectors for mashups and APIs to learn global and local semantic information about text documents at different levels of abstraction. Finally, all representation vectors output by the component are combined to achieve feature fusion and thus obtain a list of candidate APIs through the Web API recommendation component. A series of experimental results validate that SEHGN outperforms the other state-of-the-art models both in terms of overall and long-tail Web API recommendations.

In the future, we will consider the dynamic factors of Web API recommendations. As the technology updates, certain previously published Web APIs may be replaced by newer versions, and the popularity and usage patterns of certain Web APIs can also fluctuate over time. Thus, it is important to keep API recommendations with the times and relevant to the current landscape of technologies and best practices.

References

[1] D. Gannon, R. Barga, and N. Sundaresan, "Cloud-native applications," *IEEE Cloud Comput.*, vol. 4, no. 5, pp. 16–21, Sep./Oct. 2017.

[2] H. Zhao, S. Deng, Z. Liu, J. Yin, and S. Dustdar, "Distributed redundant placement for microservice-based applications at the edge," *IEEE Trans. Services Comput.*, vol. 15, no. 3, pp. 1732–1745, May/Jun. 2022.

[3] S. Roy Chowdhury, F. Daniel, and F. Casati, "Recommendation and weaving of reusable mashup model patterns for assisted development," *ACM Trans. Internet Technol.*, vol. 14, no. 2/3, pp. 1–23, 2014.

[4] J. Im, S. Kim, and D. Kim, "IoT mashup as a service: Cloud-based mashup service for the Internet of Things," in *Proc. IEEE Int. Conf. Serv. Comput.*, 2013, pp. 462–469.

[5] C. Buqing, M. Tang, and X. Huang, "CSCF: A mashup service recommendation approach based on content similarity and collaborative filtering," *Int. J. Grid Distrib. Comput.*, vol. 7, no. 2, pp. 163–172, 2014.

[6] K. Fletcher, "Regularizing matrix factorization with implicit user preference embeddings for web API recommendation," in *Proc. IEEE Int. Conf. Serv. Comput.*, 2019, pp. 1–8.

[7] B. Xia, Y. Fan, W. Tan, K. Huang, J. Zhang, and C. Wu, "Category-aware API clustering and distributed recommendation for automatic mashup creation," *IEEE Trans. Services Comput.*, vol. 8, no. 5, pp. 674–687, Sep./Oct. 2015.

[8] Y. Zhong, Y. Fan, W. Tan, and J. Zhang, "Web service recommendation with reconstructed profile from mashup descriptions," *IEEE Trans. Autom. Sci. Eng.*, vol. 15, no. 2, pp. 468–478, Apr. 2018.

[9] Q. Bao et al., "A fine-grained API link prediction approach supporting mashup recommendation," in *Proc. IEEE Int. Conf. Web Serv.*, 2017, pp. 220–228.

[10] X. Wang, H. Wu, and C.-H. Hsu, "Mashup-oriented API recommendation via random walk on knowledge graph," *IEEE Access*, vol. 7, pp. 7651–7662, 2019.

[11] G. Kang, J. Liu, B. Cao, and M. Cao, "NAFM: Neural and attentional factorization machine for web API recommendation," in *Proc. IEEE Int. Conf. Web Serv.*, 2020, pp. 330–337.

[12] Y. Xiao et al., "Structure reinforcing and attribute weakening network based API recommendation approach for mashup creation," in *Proc. IEEE Int. Conf. Web Serv.*, 2020, pp. 541–548.

[13] R. Xiong, J. Wang, N. Zhang, and Y. Ma, "Deep hybrid collaborative filtering for web service recommendation," *Expert Syst. Appl.*, vol. 110, pp. 191–205, 2018.

[14] C. Cappiello, M. Matera, M. Picozzi, F. Daniel, and A. Fernandez, "Quality-aware mashup composition: Issues, techniques and tools," in *Proc. 8th Int. Conf. Qual. Inf. Commun. Technol.*, 2012, pp. 10–19.

[15] L. Baresi, M. Garriga, and A. De Renzis, "Microservices identification through interface analysis," in *Proc. 6th IFIP WG 2.14 Eur. Conf. Serv.-Oriented Cloud Comput.*, Springer, 2017, pp. 19–33.

[16] L. Yao, X. Wang, Q. Z. Sheng, B. Benatallah, and C. Huang, "Mashup recommendation by regularizing matrix factorization with API co-invocations," *IEEE Trans. Services Comput.*, vol. 14, no. 2, pp. 502–515, Mar./Apr. 2021.

[17] T. Shen, T. Zhou, G. Long, J. Jiang, S. Pan, and C. Zhang, "DiSAN: Directional self-attention network for RNN/CNN-free language understanding," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 5446–5455.

[18] G. Letarte, F. Paradis, P. Giguère, and F. Laviolette, "Importance of self-attention for sentiment analysis," in *Proc. EMNLP Workshop BlackboxNLP: Analyzing Interpreting Neural Netw. NLP*, 2018, pp. 267–275.

[19] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, "LightGCN: Simplifying and powering graph convolution network for recommendation," in *Proc. 43rd Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2020, pp. 639–648.

[20] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural collaborative filtering," in *Proc. 26th Int. Conf. World Wide Web*, 2017, pp. 173–182.

[21] M. Shi, Y. Tang, and J. Liu, "TA-BLSTM: Tag attention-based bidirectional long short-term memory for service recommendation in mashup creation," in *Proc. Int. Joint Conf. Neural Netw.*, 2019, pp. 1–8.

[22] H. Wu, Y. Duan, K. Yue, and L. Zhang, "Mashup-oriented web API recommendation via multi-model fusion and multi-task learning," *IEEE Trans. Services Comput.*, vol. 15, no. 6, pp. 3330–3343, Nov./Dec. 2022.

[23] X. Wang, M. Xi, and J. Yin, "Functional and structural fusion based web API recommendations in heterogeneous networks," in *Proc. IEEE Int. Conf. Web Serv.*, 2023, pp. 91–96.

[24] M. Abbasi, M. Yaghoobikia, M. Rafiee, M. R. Khosravi, and V. G. Menon, "Optimal distribution of workloads in cloud-fog architecture in intelligent vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 7, pp. 4706–4715, Jul. 2021.

[25] K. Zhang, J. Tian, H. Xiao, Y. Zhao, W. Zhao, and J. Chen, "A numerical splitting and adaptive privacy budget-allocation-based LDP mechanism for privacy preservation in blockchain-powered IoT," *IEEE Internet of Things J.*, vol. 10, no. 8, pp. 6733–6741, Apr. 2023.

[26] X. Chen, Z. Zheng, and M. R. Lyu, "QoS-aware web service recommendation via collaborative filtering," in *Web Services Foundations*. Berlin, Germany: Springer, 2014, pp. 563–588.

[27] Q. Yu, Z. Zheng, and H. Wang, "Trace norm regularized matrix factorization for service recommendation," in *Proc. IEEE 20th Int. Conf. Web Serv.*, 2013, pp. 34–41.

[28] K. A. Botangen, J. Yu, Q. Z. Sheng, Y. Han, and S. Yongchareon, "Geographic-aware collaborative filtering for web service recommendation," *Expert Syst. Appl.*, vol. 151, 2020, Art. no. 113347.

[29] Y. Zhang, C. Yin, Q. Wu, Q. He, and H. Zhu, "Location-aware deep collaborative filtering for service recommendation," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 51, no. 6, pp. 3796–3807, Jun. 2021.

[30] M. M. Rahman, X. Liu, and B. Cao, "Web API recommendation for mashup development using matrix factorization on integrated content and network-based service clustering," in *Proc. IEEE Int. Conf. Serv. Comput.*, 2017, pp. 225–232.

[31] L. Yao, X. Wang, Q. Z. Sheng, W. Ruan, and W. Zhang, "Service recommendation for mashup composition with implicit correlation regularization," in *Proc. IEEE Int. Conf. Web Serv.*, 2015, pp. 217–224.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:37:05 UTC from IEEE Xplore. Restrictions apply.

[32] H. Zhao, J. Wang, Q. Zhou, X. Wang, and H. Wu, "Web API recommendation with features ensemble and learning-to-rank," in *Proc. 7th CCF Conf. Big Data*, Springer, 2019, pp. 406–419.
[33] B. Cao, X. F. Liu, M. M. Rahman, B. Li, J. Liu, and M. Tang, "Integrated content and network-based service clustering and web APIs recommendation for mashup development," *IEEE Trans. Services Comput.*, vol. 13, no. 1, pp. 99–113, Jan./Feb. 2020.
[34] M. Shi, Y. Zhuang, Y. Tang, M. Lin, X. Zhu, and J. Liu, "Web service network embedding based on link prediction and convolutional learning," *IEEE Trans. Services Comput.*, vol. 15, no. 6, pp. 3620–3633, Nov./Dec. 2022.
[35] Y. Xiao, J. Liu, R. Hu, B. Cao, and Y. Cao, "DINRec: Deep interest network based API recommendation approach for mashup creation," in *Proc. 20th Int. Conf. Web Inf. Syst. Eng.*, Springer, 2019, pp. 179–193.
[36] Q. Xue, L. Liu, W. Chen, and M. C. Chuah, "Automatic generation and recommendation for API mashups," in *Proc. 16th IEEE Int. Conf. Mach. Learn. Appl.*, 2017, pp. 119–124.
[37] F. Xie, S. Li, L. Chen, Y. Xu, and Z. Zheng, "Generative adversarial network based service recommendation in heterogeneous information networks," in *Proc. IEEE Int. Conf. Web Serv.*, 2019, pp. 265–272.
[38] C. Sang, X. Deng, and S. Liao, "Mashup-oriented web API recommendation via full-text semantic mining of developer requirements," *IEEE Trans. Services Comput.*, vol. 16, no. 4, pp. 2755–2768, Jul./Aug. 2023.
[39] S. Wu et al., "Popularity-aware and diverse web APIs recommendation based on correlation graph," *IEEE Trans. Computat. Social Syst.*, vol. 10, no. 2, pp. 771–782, Apr. 2023.
[40] W. Gong et al., "DAWAR: Diversity-aware web APIs recommendation for mashup creation based on correlation graph," in *Proc. 45th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2022, pp. 395–404.
[41] Q. Gu, J. Cao, and Y. Liu, "CSBR: A compositional semantics-based service bundle recommendation approach for mashup development," *IEEE Trans. Services Comput.*, vol. 15, no. 6, pp. 3170–3183, Nov./Dec. 2022.
[42] M. Liu, Z. Tu, H. Xu, X. Xu, and Z. Wang, "DySR: A dynamic graph neural network based service bundle recommendation model for mashup creation," *IEEE Trans. Services Comput.*, vol. 16, no. 4, pp. 2592–2605, Jul./Aug. 2023.

Xuanye Wang received the BS degree in electronic information engineering from the Zhejiang University of Technology, China, in 2022. He is currently working toward the MS degree with the Zhejiang University, Hangzhou, China. His current research interests include service computing and machine learning.

Meng Xi received the PhD degree in computer science from Zhejiang University, China, in 2022. He is currently a distinguished researcher with the School of Software Technology, Zhejiang University, Hangzhou, China. He has been a recipient of the Best Paper Award of IEEE SMDS 2020. His current research interests include service computing, data science, and machine learning.

Ying Li received the BS, MS, and PhD degrees in computer science from Zhejiang University, China, in 1994, 1997, and 2000, respectively. He is currently an associate professor with the College of Computer Science, Zhejiang University. His research interests include service computing, business process management, and compiler.

Xiaohua Pan received the master's degree in computer science and technology from Zhejiang University, in 2005, and serves as an assistant director with the Binjiang Research Institute of Zhejiang University, and a member of the Hangzhou Committee of Young Computer Science and Technology Forum of the Chinese Computer Society. His research interests include data governance, industrial internet, and service-oriented manufacturing.

Yangyang Wu received the PhD degree in computer science from Zhejiang University, China, in 2023. He is currently an assistant professor with the School of Software Technology, Zhejiang University, China. His research interests include AI enabled data governance, data imputation, and query optimization.

Shuiguang Deng (Senior Member, IEEE) received the BS and PhD degrees in computer science from Zhejiang University, China, in 2002 and 2007, respectively. He is currently a full professor with the College of Computer Science and Technology, Zhejiang University, China. He previously worked with the Massachusetts Institute of Technology in 2014 and Stanford University in 2015 as a visiting scholar. His research interests include edge computing, service computing, cloud computing, and business process management. He serves for the journal *IEEE Transactions on Services Computing*, *Knowledge and Information Systems*, *Computing*, and *IET Cyber-Physical Systems: Theory & Applications* as an associate editor. Up to now, he has published more than 100 papers in journals and refereed conferences. In 2018, he was granted the Rising Star Award by IEEE TCSVC. He is a fellow of the IET.

Jianwei Yin received the PhD degree in computer science from Zhejiang University (ZJU), in 2001. He was a visiting scholar with the Georgia Institute of Technology. He is currently a full professor with the College of Computer Science, ZJU. Up to now, he has published more than 100 papers in top international journals and conferences. His current research interests include service computing and business process management. He is an associate editor of *IEEE Transactions on Services Computing*.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:37:05 UTC from IEEE Xplore. Restrictions apply.